CAP + Vite + Cloud Foundry Deployment Guide

Project: Supplier Shield Stack: SAP CAP Node.js, SQLite, React, Vite, SAP Application Router, Cloud Foundry MTA Authentication model: Public app, no XSUAA, no IAS, no external IDP Deployment model: Packaged CAP application using mbt build and cf deploy

1. Goal

This guide explains how to build and deploy a small full stack SAP CAP application with:

- A CAP backend exposing an OData service.
- A React frontend built with Vite.
- A custom SAP Application Router serving the frontend and proxying OData calls.
- SQLite as the production database.
- No login requirement.
- Deployment to SAP BTP Cloud Foundry as one packaged MTA archive.

The final Cloud Foundry runtime shape is:

2. Important Trade-Off

SQLite on Cloud Foundry is stored on the app container filesystem. That means it is not durable storage.

Impact:

- Good for demos, prototypes, small read-heavy apps, and seed-data apps.
- The database can reset after restage, redeploy, crash recovery, or container replacement.
- Do not use this model for important production data.

For this application, the database is initialized from CSV data on application startup. That makes the app simple and self-contained.

3. Expected Project Structure

The important folders are:

4. CAP Data Model

File:

Example:

Why this exists:

- Defines the database table.
- CAP uses this model to generate SQL artifacts.
- The CSV file under db/data can seed this entity.

Impact:

- Entity name becomes part of the CSV file naming convention.
- For this namespace and entity, the seed file is:

5. CAP Service Definition

File:

Example:

Why this exists:

- Exposes the database entity as an OData service.
- Keeps the external service contract separate from the database model.
- Adds a custom function endpoint.

Impact:

- CAP exposes the service at:

- The entity collection is available at:

6. CAP Service Implementation

File:

Example:

Why this exists:

- Adds custom event handlers to the CAP service.
- Allows validation, enrichment, logging, or custom business behavior.

Impact:

- READ calls log returned supplier data.
- CREATE and UPDATE calls can be extended later.
- The custom OData function returns a string.

7. Root package.json

File:

Important configuration:

Why these changes were done:

- @cap-js/sqlite is in dependencies, not devDependencies, because it must exist in the deployed Cloud Foundry app.
- @sap/cds is in dependencies because the deployed service needs CAP runtime.
- engines.node pins the intended Node.js major version for Cloud Foundry.
- start runs cds-deploy first so SQLite is initialized when the app starts.
- cds.requires.db.kind = sqlite tells CAP to use SQLite.
- cds.requires.auth.kind = dummy avoids mandatory production authentication.

Impact:

- The CAP service can run in Cloud Foundry without HANA.
- No XSUAA service binding is required.
- No user login is required.
- The SQLite database is recreated or updated from deployment artifacts at startup.

8. React + Vite Frontend

File:

Important scripts:

Why this exists:

- Vite provides fast local development.
- vite build creates static production files in app/dist.

Impact:

- The frontend can be built independently from CAP.
- The app router can later serve the static build output.

9. Vite Development Proxy

File:

Configuration:

Why this exists:

- During local development, Vite runs on port 5173.
- CAP usually runs on port 4004.
- The proxy forwards frontend calls from /odata to CAP.

Impact:

- React code can call /odata/v4/... in both local and deployed environments.
- No hardcoded Cloud Foundry URL is needed in frontend code.

Local development commands:

In another terminal:

Then open:

10. Frontend OData Client

File:

Important pattern:

Why this exists:

- The default service root works behind the app router.
- An environment variable can override it if needed.

Impact:

- Local Vite proxy and deployed app router use the same URL path.
- The frontend remains portable across environments.

Useful checks:

The first command checks CAP directly. The second command checks Vite proxy behavior.

11. Custom Application Router

Folder:

This application uses a custom SAP Application Router because it gives direct control over:

- Static file serving.
- OData proxy routing.
- Authentication settings.
- Cloud Foundry destination wiring.

11.1 App Router package.json

File:

Configuration:

Why this exists:

- Cloud Foundry starts this as a separate Node.js app.
- @sap/approuter serves the static frontend and proxies backend calls.

Impact:

- The browser only needs to access the app router URL.
- CAP service can stay behind the app router.

11.2 App Router Routes

File:

Configuration:

Why this exists:

- /odata/* requests are forwarded to the CAP service.

- All other requests are served from static frontend files.
- authenticationType: none makes the app public.
- csrfProtection: false avoids app router CSRF handling for this public OData proxy.

Impact:

- No XSUAA is needed.
- No login page appears.
- The deployed frontend can call:

12. MTA Descriptor

File:

Final structure:

12.1 Why before-all Exists

Why:

- npm ci installs exact backend dependencies.
- npx cds build --production creates the deployable CAP output under gen/srv.
- The CSV file is copied into gen/srv so the deployed SQLite initialization can load seed data.

Impact:

- The MTAR contains the generated CAP app, not just source files.
- SQLite has seed data available inside the deployed service module.

12.2 Why There Is a suppliershield-srv Module

This module deploys the CAP backend.

Important properties:

Why:

- gen/srv is the CAP production build output.
- Cloud Foundry runs the generated CAP service as a Node.js app.

Impact:

- The backend starts with the root package start command copied into the generated build.
- The service provides a URL to other MTA modules through srv-api.

12.3 Why There Is a suppliershield-approuter Module

This module deploys the public web entry point.

Important properties:

Why:

- It packages the app router as a separate Cloud Foundry app.
- It builds the Vite frontend and copies the output into app/router/resources.

Impact:

- Users access only the app router URL.
- Static files and OData proxying are handled in one place.

12.4 Why the Destination Is Inline

Configuration:

Why:

- The app router needs to know where the CAP service is.
- The MTA provides the CAP service URL from suppliershield-srv.
- No destination service instance is required for this simple setup.

Impact:

- Fewer BTP service dependencies.
- No XSUAA or destination service binding is needed.
- The app router receives an environment destination named srv-api.

13. MTA Ignore File

File:

Recommended contents:

Why this exists:

- Keeps generated and local runtime files out of the MTAR.
- Prevents accidental packaging of large dependency folders.
- Prevents local SQLite files from being deployed by mistake.

Impact:

- Smaller deployment archive.
- Cleaner and more reproducible builds.

14. Build the Application Locally

From the project root:

Build CAP:

Build frontend:

Build MTA archive:

Expected result:

15. Deploy to Cloud Foundry

Log in:

Check target:

Deploy:

Why:

- mbt build creates the deployable MTAR package.
- cf deploy uploads and deploys the MTA to Cloud Foundry.
- -f allows updating an existing deployment.

Impact:

- Cloud Foundry creates or updates both apps:

16. Runtime Checks

Check deployed apps:

Expected:

Check routes:

Check logs:

Stream logs during troubleshooting:

Check environment variables:

Check the frontend:

Expected:

Check OData through the app router:

Expected:

or seeded supplier rows.

Check OData directly against CAP:

This verifies whether a problem is in CAP itself or in app router routing.

17. Local Development Flow

Start CAP:

Start Vite:

Open:

Check CAP:

Check Vite proxy:

18. Killing a Local Port

Find the process using port 4004:

Kill by PID:

Force kill only if needed:

For Vite:

19. Common Failure: cds Not Found

Symptom:

Cause:

- The deployed app may not have the global cds CLI.

Fix:

- Use CAP executable package scripts such as:

Why:

- CAP packages expose runtime binaries through node_modules/.bin.
- Cloud Foundry runs npm scripts with node_modules/.bin on the path.

20. Common Failure: SQLite Not Initialized

Symptoms:

or empty data when seed data was expected.

Check:

Likely causes:

- @cap-js/sqlite is missing from production dependencies.
- CSV seed files were not copied into gen/srv.
- cds.requires.db.kind is not set to sqlite.
- Start command does not run cds-deploy.

Fix:

- Keep @cap-js/sqlite in root dependencies.
- Keep the CSV copy command in mta.yaml.
- Keep root start script as:

21. Common Failure: App Router Returns 404 for OData

Check app router route:

Check direct CAP route:

If direct CAP works but app router fails, inspect:

The OData route must appear before the static catch-all route:

Why:

- Routes are evaluated in order.
- The catch-all static route would otherwise capture /odata.

22. Common Failure: Login Page Appears

Cause:

- XSUAA was added.
- A route has authentication enabled.
- CAP auth is configured for production authentication.

Checks:

Inspect:

Expected:

- No XSUAA resource in mta.yaml.
- No XSUAA binding in any module.
- authenticationType is none for public routes.
- CAP auth uses dummy mode:

Impact:

- The app is public.
- Anyone with the route can access frontend and OData endpoints.

23. Common Failure: MTAR Includes Too Much

Symptom:

- MTAR is unexpectedly large.
- Build is slow.
- Local database files are packaged.

Check:

```

```

Inspect:

```

```

Fix:

- Maintain .mtaignore.
- Do not package node_modules, local SQLite files, or old build output unless intentionally needed.

24. Clean Rebuild

Use this when a deployment behaves strangely after many local changes:

```

```

Note:

- rm -rf is destructive.
- Review the paths before running it.

25. Useful Command Cheat Sheet

Install backend dependencies:

```

```

Install frontend dependencies:

```

```

Install app router dependencies:

Run CAP locally:

Run Vite locally:

Build frontend:

Build CAP:

Build MTAR:

Deploy MTAR:

Check CF target:

Check apps:

Check one app:

Check recent logs:

Check frontend route:

Check OData through app router:

Check OData directly:

26. Deployment Checklist

Before deploy:

- package.json has @cap-js/sqlite in dependencies.
- package.json has @sap/cds in dependencies.
- package.json has start set to cds-deploy && cds-serve.
- package.json has CAP db.kind set to sqlite.
- package.json has CAP auth.kind set to dummy.
- app/router/package.json has @sap/approuter.
- app/router/xs-app.json has public routes.
- mta.yaml has only the backend and app router modules.
- mta.yaml has no XSUAA resource.
- mta.yaml has no HANA resource.
- mta.yaml copies seed CSV files into gen/srv.
- .mtaignore excludes local runtime files.

After deploy:

- cf apps shows both apps started.
- App router /index.html returns 200.
- App router /odata/v4/... returns 200.
- CAP service direct route returns 200.
- Logs do not show SQLite initialization errors.

27. Current Supplier Shield Deployment Example

The deployed app router URL from this project was:

The deployed OData endpoint was:

The generated MTAR was:

28. When to Change This Architecture

Use this SQLite and no-auth model when:

- The app is small.
- The app is a demo, prototype, or internal throwaway tool.
- Data can be reset from CSV.
- Public access is acceptable.

Switch to HANA and XSUAA or IAS when:

- Data must be durable.
- Users need login.
- Different users need different authorizations.
- The app handles sensitive or business-critical data.
- Auditing and compliance matter.